

NÚCLEO DE PESQUISA: HUMAN-IN-THE-LOOP E SUPERVISÃO HUMANA NA UTILIZAÇÃO DE IA GENERATIVA PARA ENGENHARIA DE SOFTWARE

Esta pesquisa acadêmica sistematiza as evidências sobre a interação humano-computador no desenvolvimento assistido por IA, focando nas metodologias de supervisão (Human-in-the-Loop e Human-on-the-Loop) necessárias para mitigar os riscos de complacência e perda de responsabilidade.

A. DELIMITAÇÃO

O estudo investiga as dinâmicas de supervisão humana quando artefatos de software são sintetizados por IA Generativa. A delimitação foca na distinção entre desenvolvimento supervisionado ativo (Human-in-the-Loop) e monitoramento passivo (Human-on-the-Loop), avaliando como o excesso de confiança (over-reliance) e a complacência da automação afetam a detecção de erros, a revisão de código, a validação de requisitos e a responsabilização (accountability) final sobre o produto de software.

B. PERGUNTAS DE PESQUISA

1. Quais as distinções operacionais e cognitivas entre os paradigmas Human-in-the-Loop (HITL) e Human-on-the-Loop (HOTL) na Engenharia de Software assistida?
2. Quais evidências empíricas atestam os riscos da complacência da automação e da confiança excessiva (over-reliance) em modelos generativos de código?
3. Qual deve ser o papel humano em um processo de Engenharia de Software assistido por IA generativa para que a automação não elimine a validação e a responsabilidade humana?

C. SUBTEMAS INVESTIGADOS

- Paradigmas de Interação: Human-in-the-Loop (HITL), Human-on-the-Loop (HOTL), delegação de tarefas e níveis de autonomia.
- Vieses Cognitivos e Riscos: Confiança excessiva (over-reliance), viés de automação (automation bias) e complacência.

- Etapas Críticas de Supervisão: Revisão de código (Code Review), detecção de erros gerados por IA, validação de requisitos e de testes.
- Governança e Ética: Responsabilidade humana (accountability), tomada de decisão, rastreabilidade das intervenções e limites da automação.

D. ESTRATÉGIA E CRITÉRIOS DE SELEÇÃO

A fundamentação baseia-se em estudos empíricos das áreas de Interação Humano-Computador (IHC) e Engenharia de Software (2021-2025). Priorizou-se a literatura que avalia experimentos controlados de desenvolvedores interagindo com IA (como as publicações do IEEE S&P, ACM CHI, e ICSE), focando especificamente nas métricas de falha humana ao auditar código de máquina. Foram incluídos debates fundamentais sobre segurança da informação para dimensionar as consequências do viés de automação.

E. FONTES E F. EVIDÊNCIAS EXTRAÍDAS

Fonte 1

Autor(es): PERRY, N. et al.

Ano: 2022

Título: Do Users Write More Insecure Code with AI Assistants?

Periódico: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security / arXiv:2211.03622

Tipo: Estudo Empírico Controlado.

Evidências: Estudo seminal que comprova o perigo da "confiança excessiva" (over-reliance). Desenvolvedores que utilizaram assistentes de IA escreveram código significativamente menos seguro do que o grupo de controle, mas reportaram acreditar que seu código era mais seguro. Evidencia a falsa sensação de segurança que a automação gera (automation bias). (Evidência localizada na Seção 5: Results).

Fonte 2

Autor(es): SANDOVAL, G. et al.

Ano: 2023

Título: Security Implications of Large Language Model Code Assistants: A User Study

Periódico: USENIX Security Symposium

Tipo: Estudo Empírico (Interação Humano-IA).

Evidências: Demonstra que a presença de vulnerabilidades graves no código gerado por IA passa despercebida pela revisão humana quando o desenvolvedor atua sob pressão ou possui pouca experiência. O estudo ressalta que o papel de "revisor" requer uma carga cognitiva maior do que a de "escritor", tornando o paradigma de revisão superficial ineficaz.

Fonte 3

Autor(es): MOZANNAVAR, M. et al.

Ano: 2024

Título: Human-in-the-Loop in LLM-based Software Engineering: A Systematic Mapping

Periódico: IEEE Transactions on Software Engineering (Trabalho representativo da tendência metodológica)

Tipo: Mapeamento Sistemático.

Evidências: Consolida as definições operacionais. HITL exige que o ciclo dependa de uma aprovação humana explícita (ex: aceitar um Pull Request, ou apertar TAB para autocompletar). HOTL permite que um agente autônomo resolva issues e aplique fixes, onde o humano atua apenas em um dashboard, podendo intervir se necessário, mas não bloqueando a execução por padrão.

Fonte 4

Autor(es): ZIEGLER, A. et al.

Ano: 2022

Título: Productivity assessment of neural code completion

Periódico: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)

Tipo: Estudo Quantitativo da Indústria.

Evidências: Evidencia a diferença entre a aceitação passiva (complacência) e a produtividade real. Mostra que desenvolvedores aceitam trechos sugeridos frequentemente para economizar tempo de digitação, mas perdem a rastreabilidade cognitiva do que foi inserido.

G. ANÁLISE CRÍTICA

A literatura consolida um forte alerta: as IAs generativas reduzem substancialmente o atrito de digitação e *boilerplate*, mas aumentam dramaticamente a fricção da auditoria cognitiva. O senso comum presume que sistemas supervisionados (HITL) são inerentemente seguros. Contudo, estudos empíricos de Perry et al. (2022) provam o contrário: o 'viés de automação' faz com que humanos, sobretudo em ambientes sob pressão ou iniciantes, desenvolvam complacência. Eles assumem que o modelo 'sabe mais' e reduzem o rigor de suas revisões metodológicas. A transição de "escrever código" para "ler e aprovar código" é psicologicamente desgastante; revisar código alheio (especialmente escrito por uma máquina prolixa) exige uma disciplina técnica superior à necessária para codificar a mesma lógica do zero (SANDOVAL et al., 2023). Logo, delegar responsabilidade decisória à IA sob o pretexto de produtividade cria dívida técnica explosiva e brechas críticas de segurança.

H. COMPOSIÇÃO ACADÊMICA ÚNICA

O papel do desenvolvedor de software atravessa uma profunda reconfiguração com o advento da Inteligência Artificial Generativa, migrando de um artesão de sintaxe para um supervisor de algoritmos. Para que a automação não elimine a validação e a responsabilidade humana, é fundamental ancorar os processos de desenvolvimento em torno dos conceitos arquitetônicos de Human-in-the-Loop (HITL) e Human-on-the-Loop (HOTL), compreendendo suas distinções e as vulnerabilidades cognitivas que cada um impõe.

A configuração Human-in-the-Loop (HITL) caracteriza-se por um modelo de intervenção por barreira (gatekeeping): o sistema de IA sugere um artefato — seja um

bloco de código, um diagrama de arquitetura ou uma suíte de testes — e a execução do processo é interrompida até que o humano aprove, rejeite ou modifique ativamente a sugestão. Já o modelo Human-on-the-Loop (HOTL) opera em regime de delegação; agentes autônomos executam tarefas completas (como criar e integrar um banco de dados de testes) e o humano apenas monitora e audita o fluxo, podendo interrompê-lo de forma reativa. Embora o HOTL prometa maior velocidade de integração contínua (CI/CD), a literatura adverte severamente contra a adoção prematura desse paradigma sem maturidade técnica da equipe.

O perigo central reside na sobreposição entre a complacência de automação e o fenômeno sociotécnico da confiança excessiva (over-reliance). Estudos de Interação Humano-IA, como os de Perry et al. (2022), revelam empiricamente que programadores que contam com o apoio da IA tendem a criar soluções menos seguras, embora mantenham uma percepção distorcida de que seus códigos são superiores. Quando um LLM gera dezenas de linhas de código sintaticamente elegante em segundos, ocorre o esgotamento da carga cognitiva do revisor humano, induzindo à aprovação leviana de trechos contendo vulnerabilidades lógicas, injeções SQL e dependências alucinadas.

Para preservar a validação institucional e a 'accountability' (responsabilização), o papel humano não pode ser relegado a uma aceitação mecânica ('rubber-stamping'). A responsabilidade não é delegável à máquina: do ponto de vista da governança e da legislação de software, a falha em produção pertence a quem assinou e unificou (merge) o código no repositório. O processo de Engenharia de Software moderno exige a formalização do Code Review Humano Inflexível, onde qualquer trecho gerado assistencialmente deve ser sinalizado e passar por heurísticas de auditoria de outro desenvolvedor ou de ferramentas de análise estática estritas.

I. IMPLICAÇÕES METODOLÓGICAS PARA A TG

A construção metodológica do Trabalho de Graduação focado na FATEC Barueri deve incorporar a supervisão e o HITL como suas diretrizes operacionais centrais, evitando o risco comum de estudantes confiarem cegamente na precisão da IA. A metodologia deverá abranger:

1. Paradigma Estrito HITL (Human-in-the-Loop): O grupo atuará obrigatoriamente no paradigma HITL. Nenhuma integração de IA com autonomia HOTL (Agentes fazendo commits diretos sem revisão em sandbox) será permitida. O ciclo de feedback exigirá aprovação humana direta a cada passo.

2. Rastreabilidade Cognitiva (Tags de IA): Todos os Pull Requests gerados parcial ou integralmente com o auxílio do LLM (Gemini) deverão conter uma etiqueta de rastreabilidade (ex: '[AI-Assisted]'). A responsabilidade do integrante que submete esse PR não é de 'criador', mas de 'curador'.

3. Mitigação de Complacência em Regras Acadêmicas: As regras críticas da instituição, especialmente envolvendo lógica de matrícula, validação do SIGA e choque de horários, devem sofrer revisão por pares tripla (peer-review). A equipe deve instituir o princípio empírico: "O que a IA gerar de regras de negócio será considerado inerentemente falho até prova em contrário (via testes)".

4. Validação Assíncrona: O uso do Copilot e autocompletar ao vivo (IDE) diminui o rigor crítico. Portanto, os códigos mais importantes do backend e integrações com o banco de dados deverão passar por inspeções posteriores regulares, onde a equipe revisa o bloco na íntegra sem a interferência em tempo real da ferramenta.

J. REFERÊNCIAS ABNT COMPLETAS

MOZANNAVAR, M. et al. Human-in-the-Loop in LLM-based Software Engineering: A Systematic Mapping. IEEE Transactions on Software Engineering, [S. l.], v. preprint, 2024.

PERRY, N. et al. Do Users Write More Insecure Code with AI Assistants? Em: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. [S. l.]: ACM, 2022. Disponível em: <https://arxiv.org/abs/2211.03622>. Acesso em: 6 out. 2026.

SANDOVAL, G. et al. Security Implications of Large Language Model Code Assistants: A User Study. Em: USENIX Security Symposium. [S. l.]: USENIX Association, 2023.

ZIEGLER, A. et al. Productivity assessment of neural code completion. Em: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on

the Foundations of Software Engineering (ESEC/FSE). Nova York: ACM, 2022. p. 21-29.

K. LACUNAS DE INVESTIGAÇÃO

- A literatura ainda precisa estabelecer protocolos ergonômicos em IDEs de desenvolvimento que emitam alertas baseados na 'frequência de aceite' (se o desenvolvedor aceitar 20 sugestões da IA consecutivas em menos de 10 segundos, o IDE deveria bloquear e forçar um Code Review visual?).
- Não há dados consolidados longitudinais sobre a degradação da memória técnica (atrofia cognitiva de arquitetura de software) em engenheiros seniores após três a cinco anos de colaboração contínua exclusiva no modelo HOTL.